Assignment 3: Legacy University Registration System Refactoring Using IntelliJ IDEA

1. Objective

In this assignment, students will open a deliberately messy Java application in IntelliJ IDEA, analyze its code smells, run the program, refactor it step by step, capture evidence of their work, and submit both the refactored project and a structured report.

The system simulates a university registration environment that handles students, courses, enrollment, grading, payments, transcripts, department summaries, and warning letters. The code works, but the design is poor and needs improvement without changing the intended visible behavior.

2. Learning Outcomes

- Identify common code smells in a medium-sized Java project.
- Use IntelliJ IDEA refactoring tools correctly and safely.
- Apply Extract Method, Extract Class, Move Method, Rename, Encapsulate Field, and related refactorings.
- Separate user interface logic, reporting logic, and business logic.
- Preserve behavior while improving readability, cohesion, and maintainability.
- Document a professional laboratory process with screenshots and reflections.

3. Files Included in the Starter Package

File	Purpose	Notes
Main.java	Interactive console entry point	Mixes input handling and application flow.
UniversitySystem.java	Main legacy manager class	God class with duplicated logic and long methods.
Student.java	Student model	Public fields and weak encapsulation.
Course.java	Course model	Use primitive strings and public state.
Enrollment.java	Enrollment model	Stores raw strings for grade and schedule.
Instructor.java	Instructor model	Underused and weakly modeled.
PaymentRecord.java	Payment model	Simple record with public fields.
AdminHelper.java	Legacy helper functions	Partly useful, partly inconsistent.
LegacyReportPrinter.java	Output/reporting class	Adds more design opportunities.
AuditLogger.java	Logging helper	It can be integrated or redesigned later.

4. Before Start

- Install IntelliJ IDEA Community or Ultimate Edition.
- Make sure a Java SDK is configured in IntelliJ.
- Download the "`UniversityRefactoringLab`", and extract the ZIP package into a local working folder.
- Do not edit files directly inside a compressed ZIP archive.
- Create a new screenshots folder before starting the work (You may save all your screenshots required for the assignment submission).

5. Work Procedure in IntelliJ IDEA

a. Extract and Open the Project

1. Open IntelliJ IDEA.
2. Select Open and browse to the extracted folder named UniversityRefactoringLab.
3. Open the project folder, not an individual Java file.
4. Verify that the `src` folder and all Java files are visible in the Project panel.

Screenshot 1: Project opened in IntelliJ with the src folder expanded.

b. Configure and Run the Starter Project

1. Open Main.java.
2. If IntelliJ requests an SDK, choose a valid Java version such as Java 17.
3. Click the Run icon beside the main method.
4. Test at least five menu options to understand the current behavior.
5. Record notes about what the system does and where the output appears.

Suggested quick test: add at least one student, one course, enroll the student, assign a grade, process payment, and print a transcript.

Screenshot 2: the program running in IntelliJ with the console visible.

c. Initial Analysis of the Code

1. Read `Main.java` and `UniversitySystem.java` carefully before making changes.
2. Inspect at least the methods enrollStudent, assignGrade, processPayment, printTranscript, and sendWarningLetters.
3. Identify at least 12 code smells or design problems.
4. Mark the exact files and methods where each issue occurs.
5. Prepare short notes explaining why each issue is harmful.

Examples of code smells include god class, duplicated loops, public fields, magic numbers, repeated email checks, long methods, primitive obsession, mixed input/output and business logic, poor cohesion, and weak encapsulation.

Screenshot 3: at least six screenshots of problematic code sections in the original version.

d. Create a Refactoring Plan

1. Create a short planning document in Word, PDF, or Markdown.
2. List the problems to be addressed in order.
3. State the refactoring technique for each problem.
4. Explain the expected benefit and any risk.
5. Group changes into safe stages instead of trying to rewrite everything at once.

A strong sequence is (example): safe renaming first, small method extraction second, repeated logic cleanup third, structural extraction fourth, and deeper model improvement last.

Required Information: planning file or notes visible in IntelliJ or as a separate document.

6. Required Refactoring Stages

Stage 1: Safe cleanup

Rename confusing variables, parameters, and methods; format the code; replace obviously bad names with meaningful ones.

You may use IntelliJ's auto-detector, such as a magic number or a name.

Stage 2: Method extraction

Break long methods into smaller ones, such as finding a student, finding a course, checking prerequisites, checking schedule conflict, calculating the fee, validating email, and printing summaries.

Stage 3: Remove duplication

Reduce repeated loops and repeated email-checking logic; centralize repeated logic where practical.

Stage 4: Improve encapsulation

Begin converting public fields into private fields with getters, setters, or behavior methods where appropriate.

Stage 5: Separate concerns

Move printing/reporting logic out of UniversitySystem where possible; reduce business logic inside Main.java.

Stage 6: Improve domain design

Introduce enums or constants for student type, academic status, payment method, and grade, if applicable.

Stage 7: Structural refactoring

Extract classes such as EnrollmentService, PaymentService, TranscriptService, NotificationService, or FeeCalculator.

Stage 8: Final review

Run the application again, compare outputs, clean formatting, and verify that the program still works.

[These stages are just suggestions. If you can come up with different solutions or identify further code smells and rectify them, it would contribute to a higher grade.]

7. Minimum Required Refactoring Techniques (Suggestions)

- Rename Method
- Rename Variable or Parameter
- Extract Method
- Extract Class
- Move Method
- Encapsulated Field
- Replace Magic Numbers with Constants
- Remove Duplicate Code
- Decompose Conditional
- Separate Presentation Logic from Business Logic

You must apply at least 8 techniques in total, and at least 3 of those must be structural improvements such as Extract Class, Move Method, or service extraction.

8. Deliverables

- Original starter project folder.
- Refactored project folder [include all the Java files].
- Refactoring report in PDF or DOCX.
- Screenshots folder.

9. Suggested Report Structure

- Introduction
- Initial system understanding
- Code smells identified
- Refactoring plan
- Step-by-step refactoring evidence
- Final design summary
- Reflection
- [Do not include complete code, rather point to the refactoring codes]

10. Academic Integrity and Anti-Copy Requirements

This assignment is designed to reduce copied AI submissions. Students must provide process evidence showing that the work was done in IntelliJ and in small steps [screenshots].

- Students must explain their own decisions in their own words.
- Screenshots must match the submitted code state.
- Commit messages or checkpoints must show progression.
- Students may be asked to explain one refactoring live.
- Code that appears fully rewritten in one jump may receive reduced credit if the process evidence is weak.

11. Note for the final code

You are not expected to produce one identical final architecture. [You may have your unique final code]

Different good solutions are acceptable as long as the design is improved, the behavior remains sensible, and the refactoring process is documented.

An excellent final structure may include classes such as EnrollmentService, PaymentService, TranscriptService, NotificationService, and FeeCalculator, as well as stronger model classes that use encapsulation.

Criteria	Excellent (A)	Good (B)	Satisfactory (C)	Poor (D/F)	Marks
1. Code Smell Identification (15%)	Identifies 9+ meaningful code smells with clear explanations and examples from code	Identifies 6–8 relevant code smells with reasonable explanations	Identifies 4–5 basic code smells with limited explanation	Identifies fewer than 4 or incorrect/unclear issues	/15
2. Refactoring Plan (10%)	Clear, structured, logical sequence with justification, risks, and expected outcomes	Mostly clear plan with some justification and logical order	Basic plan, limited reasoning, weak sequencing	No clear plan or poorly structured plan	/10
3. Use of IntelliJ Refactoring Tools (10%)	Excellent use of IntelliJ tools (rename, extract, move, etc.) with clear evidence in screenshots	Good use of IntelliJ tools with minor gaps	Some use of tools, but mostly manual changes	Little or no use of IntelliJ refactoring tools	/10
4. Step-by-Step Refactoring Process (20%)	Detailed step-by-step refactoring with before/after screenshots and clear explanations	Most steps are documented with screenshots and explanations	Some steps documented but incomplete or unclear	Missing steps, screenshots, or explanations	/20
5. Refactoring Techniques Applied (15%)	Applies 6+ techniques, including advanced ones (Extract Class, Move Method, etc.), correctly	Applies 4–5 techniques correctly	Applies 2–4 basic techniques only	Applies fewer than 2 or incorrect use	/15
6. Code Quality Improvement (15%)	Significant improvement: reduced duplication, better structure, clear separation of concerns	Good improvement with noticeable design enhancement	Some improvement, but still many design issues	Minimal or no improvement	/15
7. Final System Design (10%)	Well-structured design with clear separation (services, models, etc.) and good naming	Improved structure with some separation	Basic improvements, but still tightly coupled	Poor structure remains unchanged	/10
8. Documentation & Report Quality (5%)	Very clear, well-organized, professional report with strong explanations	Clear report with minor issues	Basic report, lacks clarity in places	Poorly written or incomplete report	/5
9. Reflection & Justification (5%)	Deep, thoughtful reflection with a strong understanding of design and challenges	Good reflection with reasonable insight	Basic reflection with limited depth	Weak or missing reflection	/5
10. Authenticity Evidence (Screenshots, Process, Commits) (5%)	Strong evidence: consistent screenshots, clear progression, believable workflow	Good evidence, but minor gaps	Limited evidence of process	Little or no evidence (possibly copied work)	/5